TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 8

Workloads and Scheduling

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 8: Workload types and scheduling 3

8.1 Choosing a workload controller 3

8.2 How scheduling works 4

8.3 Requests, limits and QoS 6

8.4 Live lab: a 3-node cluster 6

Class 8 - Top 20 interview questions 9

K8S · CLASS 08 · 60-90 minutes

Workload types and scheduling

By the end you can choose the right workload controller and control exactly where pods run and how much CPU and memory they get.

What you will learn today

- DaemonSet, Job and CronJob - and when to use each controller
- How the scheduler picks a node
- nodeSelector, node affinity, pod affinity and anti-affinity
- Taints and tolerations; cordon and drain
- Resource requests and limits, QoS classes, OOMKilled
- PriorityClass, preemption and topology spread
- Lab: 3-node cluster - place, block and evict pods

Part	Topic	Time
1	Story and workload types	15 min
2	Scheduling rules	20 min
3	Resources and QoS	15 min
4	Live lab	20 min
5	Recap and interview questions	10 min

8.1 Choosing a workload controller

Ravi's story

Three new requests reached Ravi in one week: run a log collector on **every** server, run a database cleanup **once**, and generate a sales report **every night at 2 AM**. A Deployment fits none of them. Kubernetes has a controller for each.

Controller	Runs	Typical use	Everyday example
Deployment	N interchangeable pods, forever	Web apps, APIs	Cooks on every shift
StatefulSet	N pods with identity and storage	Databases, Kafka	Numbered train seats
DaemonSet	One pod on each (matching) node	Log/monitoring agents, CNI, kube-proxy	A security guard on every floor
Job	Pods until a task completes	Migrations, batch processing	A one-time deep cleaning
CronJob	A Job on a schedule	Backups, reports, cleanups	A daily alarm clock

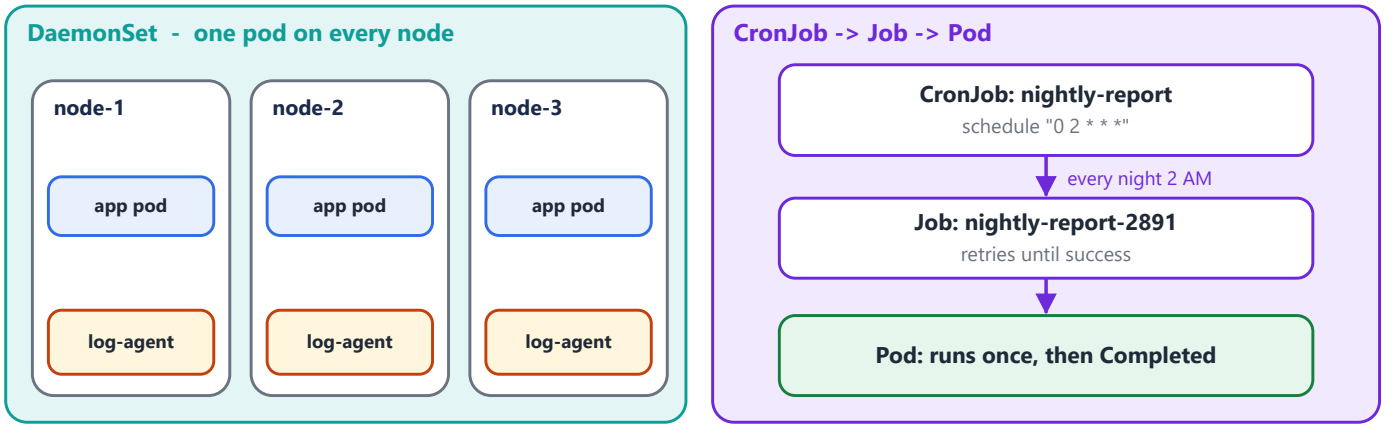


Figure 8.1 - A DaemonSet runs one pod per node; a CronJob creates Jobs on a schedule

cronjob.yaml

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-report
spec:
  schedule: "0 2 * * *"            # min hour day month weekday
  concurrencyPolicy: Forbid        # Allow | Forbid | Replace
  successfulJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 4              # retries before marking failed
      completions: 1
      parallelism: 1
      activeDeadlineSeconds: 600
      ttlSecondsAfterFinished: 3600
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: report
              image: busybox:1.36
              command: ["sh", "-c", "echo Sales report generated at $(date)"]
```

8.2 How scheduling works

For every new pod, the scheduler **filters** out nodes that cannot run it (not enough free requests, wrong labels, taints, port conflicts, volume zone) and then **scores** the rest (spread, balanced usage, affinity preferences). The highest-scoring node wins and the pod is bound to it. If no node passes the filter, the pod stays **Pending**.

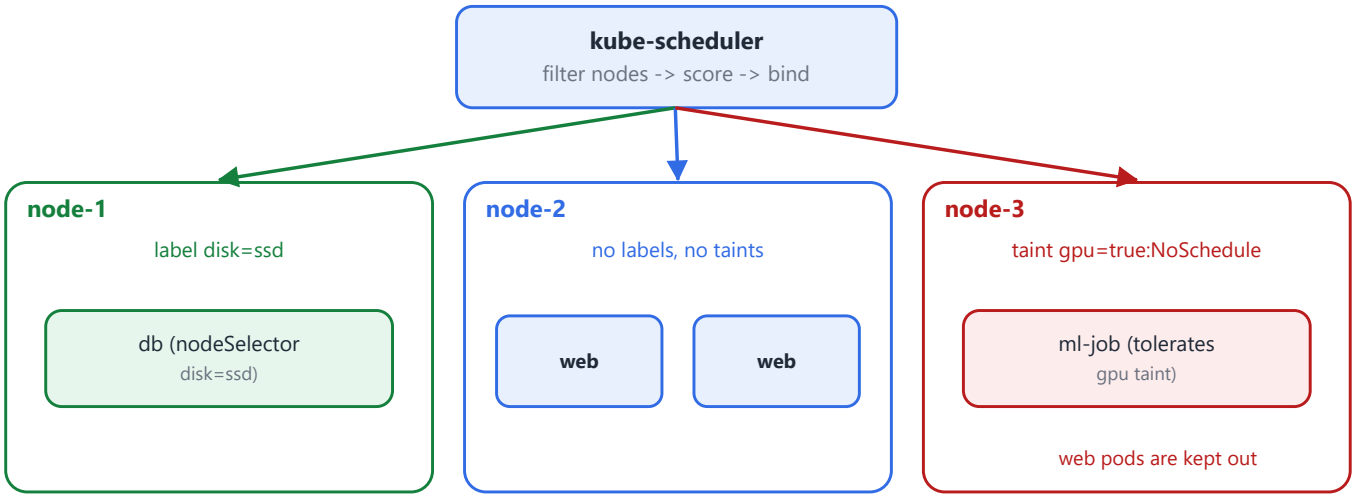


Figure 8.2 - nodeSelector attracts pods; taints repel pods without a toleration

Tool	Direction	Example
nodeSelector	Pod must land on nodes with a label	`disk: ssd`
Node affinity	Like nodeSelector, with operators and required/preferred rules	requiredDuringScheduling... `zone In [a, b]`
Pod affinity	Place near other pods	Put cache next to the web app
Pod anti-affinity	Keep away from other pods	Spread 3 replicas over 3 nodes for HA
Taint (on node)	Node repels pods	`gpu=true:NoSchedule`
Toleration (on pod)	Pod is allowed on a tainted node	`key: gpu, effect: NoSchedule`
topologySpreadConstraints	Spread evenly across zones/nodes	`maxSkew: 1` per zone

Everyday example
Taints and tolerations work like a **VIP lounge** at an airport. The lounge (node) has a sign "members only" (taint). Only passengers with a lounge card (toleration) may enter. But having a card does not force you into the lounge - to force placement you also need affinity or nodeSelector.

Taint effect	Behaviour
NoSchedule	New pods without a toleration are not scheduled here
PreferNoSchedule	The scheduler tries to avoid the node, but may use it
NoExecute	New pods are blocked and existing pods without a toleration are evicted

Cordon marks a node unschedulable (existing pods stay). **Drain** cordons it and safely evicts its pods - used before maintenance or upgrades. **Uncordon** brings it back.

8.3 Requests, limits and QoS

```
resources:
  requests:            # reserved; used by the scheduler
    cpu: 250m          # 250 millicores = 1/4 CPU
    memory: 128Mi
  limits:              # hard ceiling enforced on the node
    cpu: "1"
    memory: 256Mi
```

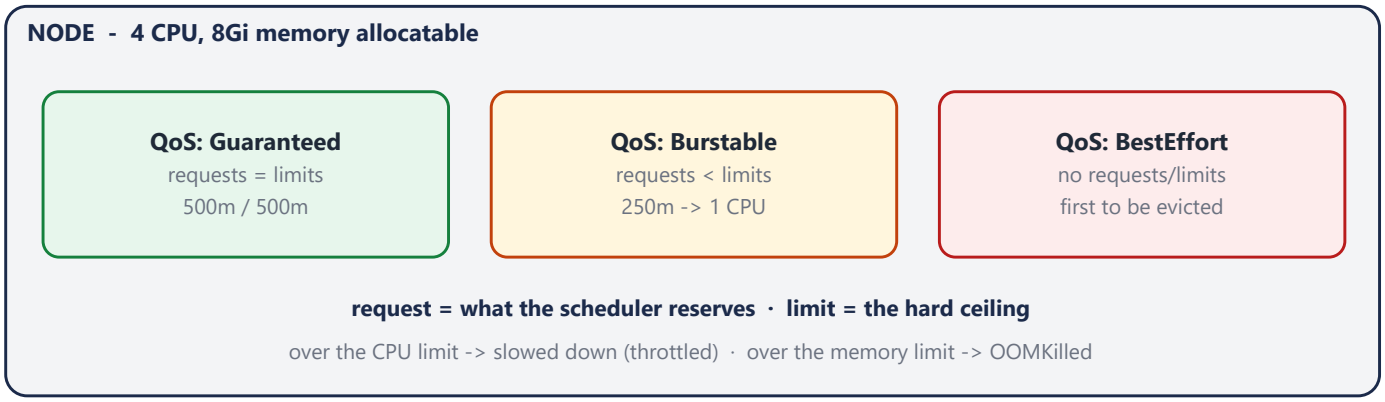


Figure 8.3 - Requests, limits and the three QoS classes

- Over the **CPU limit**: the container is **throttled** (slowed down), never killed.
- Over the **memory limit**: the container is **OOMKilled** (exit code 137) and restarted.
- When a node runs short of memory, the kubelet **evicts** pods: BestEffort first, then Burstable, Guaranteed last.

Everyday example
Booking a restaurant table for 4 is the **request** - the restaurant reserves those seats even if only 2 come. The fire-safety limit of 6 people per table is the **limit**. If a seventh person squeezes in, the manager asks someone to leave (OOMKilled).

PriorityClass gives pods a priority number. When the cluster is full, the scheduler may **preempt** (evict) lower-priority pods to make room for critical ones, such as payment over a nightly report.

8.4 Live lab: a 3-node cluster

ds.yaml

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-agent
spec:
  selector:
    matchLabels: { app: node-agent }
  template:
    metadata:
      labels: { app: node-agent }
    spec:
      containers:
        - name: agent
          image: busybox:1.36
          env:
            - name: NODE_NAME
              valueFrom: { fieldRef: { fieldPath: spec.nodeName } }
          command: ["sh", "-c", "while true; do echo collecting logs on $NODE_NAME; sleep 10; done"]
```

db-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: db
spec:
  nodeSelector:
    disk: ssd
  containers:
    - name: db
      image: nginx:1.27
```

PowerShell - workloads

```
minikube start -p lab --nodes 3 --driver=docker
kubectl get nodes                               # lab, lab-m02, lab-m03

# DaemonSet: one pod per node
kubectl apply -f ds.yaml
kubectl get pods -o wide -l app=node-agent      # one per node
kubectl logs -l app=node-agent --prefix

# Job and CronJob
kubectl create job hello --image=busybox:1.36 -- echo "cleanup done"
kubectl get jobs; kubectl logs job/hello
kubectl apply -f cronjob.yaml
kubectl create job manual-run --from=cronjob/nightly-report
```

PowerShell - scheduling

```
# nodeSelector
kubectl apply -f db-pod.yaml
kubectl get pod db                              # Pending - no node has disk=ssd yet
kubectl label node lab-m02 disk=ssd
kubectl get pod db -o wide                      # now Running on lab-m02

# taints
kubectl taint node lab-m03 gpu=true:NoSchedule
kubectl create deployment web --image=nginx:1.27 --replicas=6
kubectl get pods -o wide -l app=web             # none on lab-m03

# drain for maintenance
kubectl drain lab-m02 --ignore-daemonsets --delete-emptydir-data --force
kubectl get pods -o wide                        # pods moved away
kubectl uncordon lab-m02
kubectl taint node lab-m03 gpu=true:NoSchedule-   # remove taint
```

oom.yaml - try: kubectl apply, get pod -w, describe

```
apiVersion: v1
kind: Pod
metadata:
  name: memory-hog
spec:
  containers:
    - name: stress
      image: polinux/stress
      resources:
        limits: { memory: 100Mi }
        requests: { memory: 50Mi }
      command: ["stress"]
      args: ["--vm", "1", "--vm-bytes", "250M", "--vm-hang", "1"]
```

You will see **OOMKilled**, then CrashLoopBackOff. `kubectl describe pod memory-hog` shows `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`. Clean up with `minikube delete -p lab`.

Common mistake

- Setting no requests: the scheduler thinks the pod needs nothing and overpacks nodes.
- Setting CPU limits too low for Java apps: start-up becomes painfully slow due to throttling.
- Expecting a toleration to *attract* a pod - it only *permits*; add affinity too.

Class 8 recap

- DaemonSet = one per node; Job = run to completion; CronJob = scheduled Job.
- Scheduler: filter then score. Pending = no node passed the filter.
- nodeSelector/affinity attract; taints repel; tolerations permit.
- Requests reserve, limits cap. CPU throttles, memory OOMKills.
- QoS: Guaranteed > Burstable > BestEffort.

Class 8 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is a DaemonSet? Give examples.

A controller that runs one copy of a pod on every node, or every node matching a selector. New nodes automatically get the pod. Examples: log collectors (Fluent Bit), monitoring agents (node-exporter), kube-proxy and CNI agents.

Q2. What is a Job?

A controller that runs pods until a specified number complete successfully. It retries failed pods up to backoffLimit (default 6). Used for batch tasks and migrations.

Q3. What is a CronJob?

A controller that creates Jobs on a cron schedule, such as "0 2 * * *" for 2 AM daily, with settings for concurrency, history limits and deadlines.

Q4. Explain completions and parallelism in a Job.

completions is how many successful pods are needed; parallelism is how many may run at once. completions 10 and parallelism 3 processes ten items three at a time.

Q5. What does concurrencyPolicy do in a CronJob?

It controls overlapping runs: Allow (default) lets them overlap, Forbid skips a new run if the previous one is still running, Replace stops the old run and starts the new one.

Q6. How does the kube-scheduler decide where to place a pod?

It filters nodes that satisfy requirements (resources, nodeSelector, affinity, taints, ports, volume topology), scores the remaining nodes by priorities such as spreading and balanced usage, and binds the pod to the highest-scoring node.

Q7. What is nodeSelector?

The simplest placement constraint: a map of labels that a node must have for the pod to be scheduled there.

Q8. Node affinity vs nodeSelector?

Node affinity is more expressive: operators like In, NotIn, Exists, Gt, and both hard (requiredDuringSchedulingIgnoredDuringExecution) and soft (preferred...) rules with weights.

Q9. What are pod affinity and anti-affinity?

Rules based on labels of other pods. Affinity co-locates pods (cache with app); anti-affinity separates them, commonly to spread replicas across nodes or zones using topologyKey.

Q10. What are taints and tolerations?

A taint on a node repels pods; a toleration on a pod allows it onto a node with a matching taint. They are used to dedicate nodes, for example GPU or control-plane nodes.

Q11. What are the taint effects?

NoSchedule blocks new pods, PreferNoSchedule avoids the node if possible, and NoExecute also evicts existing pods that do not tolerate the taint.

Q12. Taints vs node affinity?

Taints are node-centric and keep unwanted pods away; affinity is pod-centric and attracts pods to nodes. To dedicate nodes to specific pods you combine both.

Q13. What is the difference between cordon and drain?

Cordon marks a node unschedulable but leaves existing pods. Drain cordons and evicts pods (respecting PodDisruptionBudgets) so maintenance can be done. Uncordon makes it schedulable again.

Q14. What are resource requests and limits?

Requests are the CPU/memory guaranteed and used by the scheduler for placement. Limits are the maximum a container may use, enforced by the kubelet and Linux cgroups.

Q15. What happens when a container exceeds its CPU or memory limit?

Exceeding CPU leads to throttling. Exceeding memory makes the kernel kill the container (OOMKilled, exit code 137) and the kubelet restarts it according to the restart policy.

Q16. What are QoS classes?

Guaranteed (every container has requests equal to limits for CPU and memory), Burstable (some requests/limits set but not equal) and BestEffort (none set). Under memory pressure BestEffort is evicted first and Guaranteed last.

Q17. What does 250m CPU mean?

250 millicores - a quarter of one CPU core. Memory uses binary units such as Mi and Gi.

Q18. What is a PriorityClass and preemption?

A PriorityClass assigns an integer priority to pods. If a high-priority pod cannot be scheduled, the scheduler can preempt (evict) lower-priority pods to make room.

Q19. Why would a pod stay in Pending?

Insufficient CPU/memory on all nodes, unmatched nodeSelector or affinity, untolerated taints, an unbound PVC, quota limits or no ready nodes. `kubectl describe pod` shows the scheduler's reason in Events.

Q20. What are topologySpreadConstraints?

Rules that spread pods evenly across topology domains such as zones or nodes, using maxSkew, topologyKey and whenUnsatisfiable - improving high availability.